Comms report for handoff

1. Project Overview

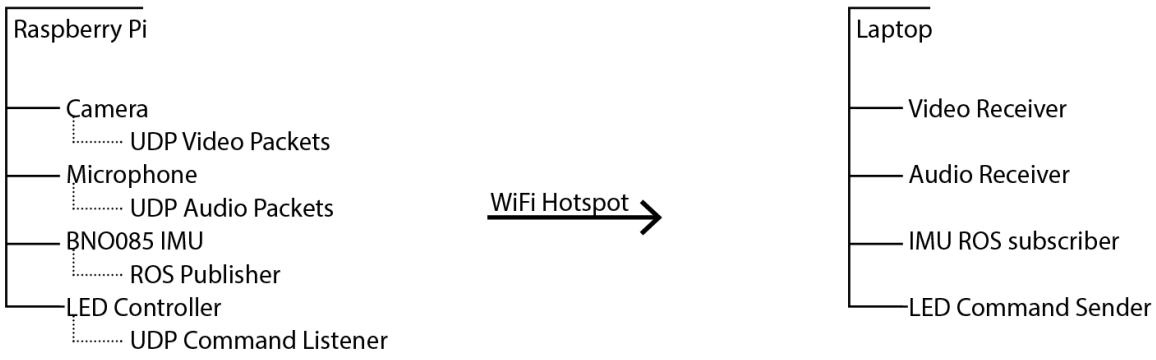
Goal: Create a low-latency communication system between a Raspberry Pi robot and a remote operator laptop capable of

- Live video streaming
- Live audio streaming
- IMU telemetry
- LED control commands

System Capabilities as of October 6th

Component	Status
Video Stream	Working
Audio Stream	Working
IMU Telemetry	Working
LED Control	Working
ROS integration	IMU only

2. System Architecture



3. Networking Setup

Hardware

- Raspberry Pi model
- Laptop
- IMU model (BNO085)
- Camera (VD56G3)

Network

On hotspot:

- Raspberry Pi: 172.20.3
- Laptop: 172.20.2

Ports

Service	Port
IMU	6000
Video	6009
Video commands	6002
Audio	6009
Led	6010

4. Software components

IMU System (orientation)

BNO085 → IMUstream.py → UDP imu_bridge.py → ROS2 Topic → /imu/data

Both `IMUstream.py` and `imu_bridge.py` run on the Raspberry Pi. `IMUstream.py` reads the BNO085 sensor and sends the measurements over UDP to the local Pi network interface. `imu_bridge.py` receives the data and publishes it as a ROS 2 `sensor_msgs/Imu` message. ROS 2 runs inside Docker on the laptop, where the `/imu/data` topic can be monitored with `ros2 topic echo`.

Library	Purpose
Socket	UDP communication
Board	Raspberry Pi hardware access
Bitbangio	I2C communication with BNO085
Adafruit BNO08x	BNO085 sensor interface
Rclpy	ROS 2 Python interface
sensor_msgs/Imu	Standard ROS 2 IMU message

PSEUDO CODE:

IMUstream.py

```
Initialize BNO085

Enable acceleration, gyroscope, magnetometer, rotation vector

Loop at 20 Hz:

      Read sensor data

      Pack values into UDP message

      Send packet
```

imu_bridge.py

```
Initialize ROS 2 node

Create /imu/data publisher

Bind UDP socket to port 6000

Loop:

      Receive UDP packet

      Parse sensor values

      Create Imu message

      Set timestamp and frame_id

      Publish to /imu/data
```

Decisions

- Used lightweight UDP for continuous sensor data where receiving the latest measurement is more important than retransmitting dropped packets.
- Used `qos_profile_sensor_data` to prioritize current IMU measurements.

Issues Encountered

Rclpy Version Mismatch

- **Symptom:** `ModuleNotFoundError: No module named 'rclpy'`
- **Cause:** Python environment mismatch.
- **Resolution:** Run the ROS bridge from the ROS-compatible environment inside the Docker/ROS setup.

Hotspot IP Changes

- **Symptom:** No IMU packets received.
- **Cause:** Laptop IP changed when reconnecting to the phone hotspot.
- **Resolution:** Update the target IP in `IMUstream.py`; check the Pi with `hostname -I` and the laptop with `ipconfig`.

Video + Audio

CSI Camera → Picamera2→ m_camera_udp_commands.py –(laptop side)-- → media_receiver.py → Laptop (+commands) → OpenCV

Bluetooth mic → Pipewire (pipe host)→ m_audio_stream.py–(laptop side)-- → media_receiver.py → laptop speaker

Library	Purpose
Picamera2	CSI camera capture
OpenCV (cv2)	Image conversion, JPEG compression, display
PipeWire (pw-record)	Audio capture from Bluetooth microphone
socket	UDP communication
struct	Packet serialization and headers
sounddevice	Audio playback on laptop
threading	Concurrent receive/playback/control threads
queue	Buffer management for real-time streaming

PSEUDO CODE:

m_camera_udp_commands.py

```
Python

Initialize camera

Start command listener thread

Loop:

	Capture frame

	Convert frame to OpenCV format

	Compress frame as JPEG

	Split JPEG into UDP packets

	Add timestamps and frame metadata Send packets to laptop

	Maintain target FPS
```

m_audio_stream.py

```
Start PipeWire audio capture

Loop:

    Read audio samples

    Buffer samples into fixed-size chunks

    Add timestamp and sequence number

    Send UDP packet to laptop
```

media_receiver.py

```
Start UDP receiver

Create audio and video queues

Receiver Thread:

    Receive packet

    If audio: Store newest audio packet

    If video: Reconstruct JPEG frame

        Store newest complete frame

Audio Thread:

    Play audio packets

Main Thread:

    Decode JPEG frames

    Display video

Command Thread:
```

```
Send camera commands
```

Camera Commands:

- START
- STOP
- SET_BRIGHTNESS
- SET_FPS
- SET_QUALITY
- SET_EXPOSURE

Decisions

- UDP chosen instead of TCP to minimize latency.
- JPEG compression used to reduce network bandwidth.
- Video frames split into multiple UDP packets before transmission.
- Audio and video consolidated into a single receiver application.
- Bounded queues prioritize newest data and discard stale packets when processing falls behind.
- Common packet headers added to support multiple media types over the same port.

Issues Encountered

- Initial implementations used separate receivers for audio and video, making synchronization and deployment more difficult.
- ROS-based audio streaming was tested successfully, but maintaining both audio and video through ROS added unnecessary complexity for the final communication system.
- Large packet sizes caused instability and packet fragmentation; packet sizes were reduced and tuned during testing.
- Buffer growth increased latency significantly; bounded queues were introduced to prioritize real-time performance over complete delivery.

LED control (turn on and off from remote operator)

Laptop → UDP Command (ON/OFF) → Raspberry Pi Listener → NeoPixel Driver → LED Strip

Library	Purpose
Socket	UDP communication between laptop and Raspberry Pi
Board	Access Raspberry Pi GPIO pins (board.D18)
Neopixel	Control NeoPixel LED strip

PSEUDO CODE:

LED_server.py

```
Python

Initialize NeoPixel strip

Turn LEDs OFF at startup

Create UDP server on port 6010

Loop:

    Receive command

    If ON: Turn LEDs white

    If OFF:Turn LEDs off

On exit:

    Turn LEDs off

    Close socket
```

led_control.py

```
Python

Create UDP client
```

```
Loop:
    Read user input
    If ON:Send ON command
    If OFF:Send OFF command
    If Q:Exit
Close socket
```

Issues Encountered:

- Script must be executed directly from the terminal on the Raspberry Pi.
- NeoPixel access requires elevated permissions (`sudo`) because it interfaces directly with GPIO hardware.
- IP address must match the Raspberry Pi's current network address.

5. Performance Evaluation

- Video: `~3.3 s latency Stable`
- Audio: `~3.3 s latency Stable`
- IMU: `Near real-time telemetry`
- LED: `Command latency not perceptible`

6. Things That Did Not Work

This is one of the most useful sections.

Future students won't repeat dead ends.

Example:

ROS Video Streaming

Attempted: `ROS image transport`

Result: `Added complexity, No benefit for current use case`

TCP Streaming

Attempted: `TCP sockets`

Result: `Higher latency`

Firewall Changes

Attempted: `Disable Windows Firewall`

Result: Sometimes it is the issue

7. Known Limitations

- Video latency ~3.3 s
- No packet retransmission
- IP addresses currently hardcoded
- Hotspot changes may require updates
- No automatic reconnection

8. List of files:

File on how to run this pipeline **`usage.md`**

File on how to transfer environments to laptop or pi: **`environment transfer.md`**

Environments:

- **`requirements.txt`** (pi environment)
- **`ros_env1.yam`**l (laptop anaconda environment)
- **`ros_test.tar`** (pi docker image for ROS in the pi)

Extra notes while developing:

> ROS2 works in the raspi via docker for a listener/talker test steps on **`ros notes.txt`**
>
> Version of audio being transmitted with ROS2 steps: **`Audio pipeline.txt`**

WORKING FILES:

> PI:
>
> - IMUstream.py
> - imu_bridge.py
> - m_audio_stream.py
> - m_camera_udp_commands.py
> - led_server.py
>
> Laptop:
>
> - media_receiver.py
> - led_control.py